

Informe de Orientación del Semi-proyecto: Monitorización de Redes Sociales para Análisis de Sentimientos

Descripción del Proyecto

¿Qué se pretende lograr?

El objetivo central de este proyecto es analizar millones de posts/tweets en tiempo real para medir la opinión pública sobre diversas entidades, mediante un pipeline de procesamiento de datos robusto y escalable utilizando tecnologías de Big Data.

Dataset Seleccionado

- **Nombre:** Twitter-Entity Sentiment Analysis
- **Fuente:** Kaggle (se utilizan `twitter_training.csv` y `twitter_validation.csv` para entrenar las distribuciones de datos sintéticos).
- **Formato:** CSV

Justificación del Dataset

Este dataset es ideal para el proyecto por las siguientes razones:

- **Volumen:** Aunque los archivos originales (`twitter_training.csv` con $\sim 74k$ registros y `twitter_validation.csv` con $\sim 1.2k$ registros) no son masivos por sí solos, proporcionan una base estadística sólida. Para simular “Grandes Volúmenes de Datos”, se ha implementado un generador de datos sintéticos que replica las distribuciones de este dataset y puede emitir tweets a Kafka de forma continua y a gran escala.
- **Características:** Contiene atributos cruciales como `Tweet ID`, `Entity` (la marca o tema al que se refiere el tweet), `Sentiment` (clasificado como `Positive`, `Negative`, `Neutral`, `Irrelevant`) y `Tweet content`. La presencia de ruido (caracteres especiales, URLs, etc.) también es una característica realista que el pipeline debe manejar.
- **Pertinencia:** Se relaciona directamente con el objetivo del proyecto, ya que proporciona datos ya clasificados por sentimiento y entidad, lo que permite enfocar el esfuerzo

en la infraestructura de procesamiento y visualización.

Arquitectura Propuesta

Nuestro enfoque combinará procesamiento **Batch** y **Streaming** para abordar las diferentes necesidades del análisis de sentimientos en redes sociales.

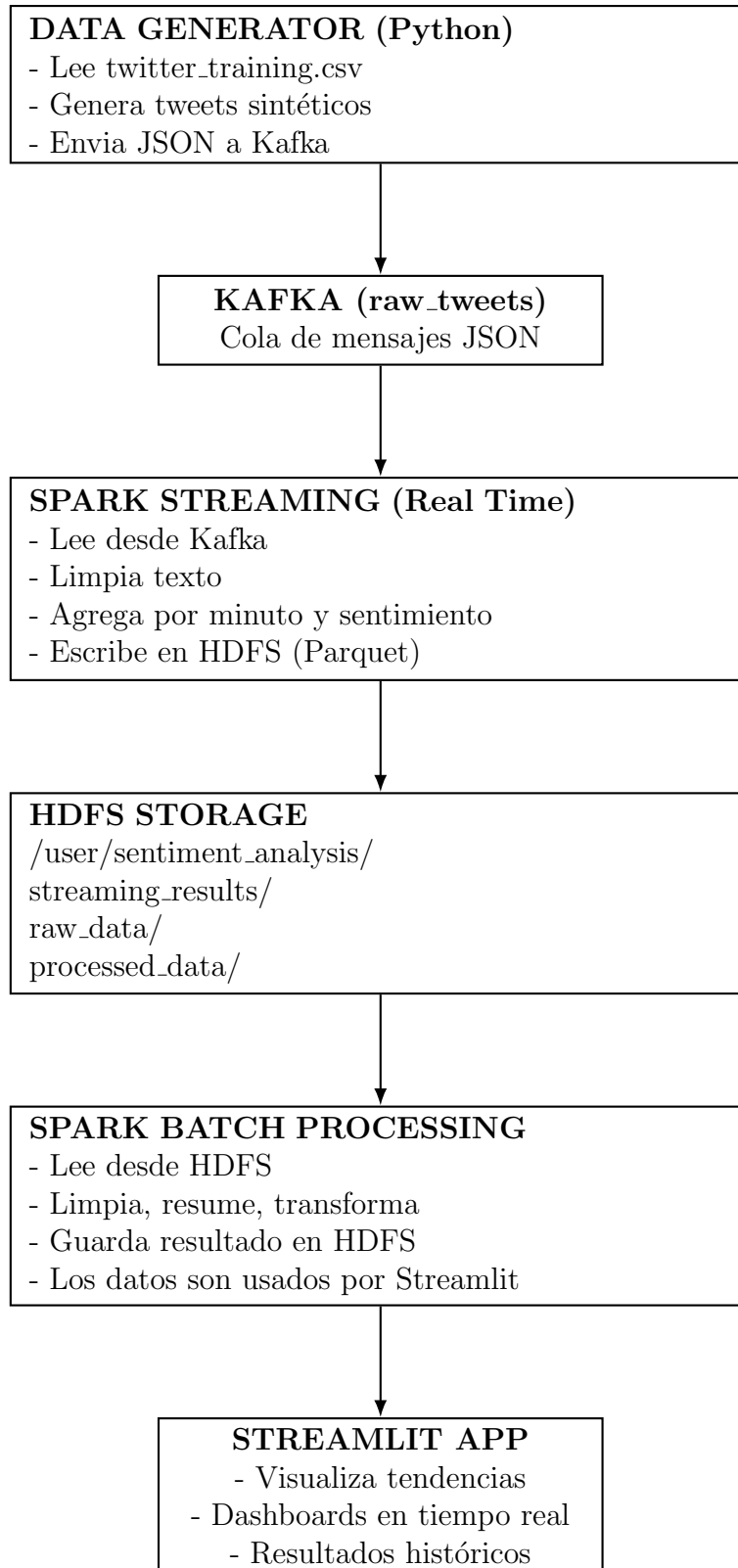


Figure 1: Flujo detallado del pipeline de procesamiento propuesto.

Componentes y Explicación

- **Data Generator (Python/Kafka-Python):** Simula un flujo continuo de tweets sintéticos, basándose en las distribuciones estadísticas del dataset original. Publica estos tweets como mensajes JSON en un topic de Kafka. (*Streaming*)
- **Kafka Broker:** Sistema de mensajería distribuido que recibe los tweets del Data Generator y los pone a disposición de los consumidores. (*Streaming*)
- **Spark Streaming (PySpark):** Consume tweets en tiempo real desde Kafka, realiza limpieza de texto y agregaciones en ventanas de tiempo. Los resultados se almacenan en HDFS. (*Streaming*)
- **HDFS (Hadoop Distributed File System):** Almacena los datos brutos y los resultados agregados del procesamiento. También guarda checkpoints. (*Batch y persistencia*)
- **Spark Batch (PySpark):** Procesa un subset inicial del dataset CSV en HDFS. Realiza limpieza y transformación básica. (*Batch*)
- **Streamlit Dashboard (Python):** Interfaz interactiva para visualizar datos en tiempo real desde Kafka y datos procesados desde HDFS. (*Lectura de streaming y batch*)

Avances

Se han logrado los siguientes avances, demostrando un prototipo funcional:

Instalación y configuración de la infraestructura

- Clúster Docker Compose con HDFS, Spark, Kafka y Zookeeper.
- Comunicación entre contenedores y servicios operativos.
- Configuración de `SparkSession` para conexión con Spark y HDFS.

Carga de un subset inicial del dataset en HDFS

- Archivo `twitter_training.csv` copiado a `/user/sentiment_analysis/raw_data/`.
- Usado por `data_generator.py` para calcular distribuciones estadísticas.

Procesos básicos de limpieza y transformación

Procesamiento Batch (`spark-app.py`):

- Carga del CSV desde HDFS.
- Renombrado de columnas.
- Eliminación de valores nulos.

- Limpieza de texto (minúsculas, caracteres especiales).
- Guardado en formato Parquet.

Procesamiento Streaming (`spark_streaming.py`):

- Consumo de mensajes JSON desde Kafka.
- Parseo y conversión de timestamp.
- Limpieza de texto.
- Agregación por sentimiento en ventanas de 1 minuto.
- Escritura continua en HDFS con checkpoint.

Pruebas iniciales de consultas

En Batch:

- Impresión de esquema y primeros registros.
- Conteos agrupados por `sentiment` y `entity`.
- Verificación de lectura desde Parquet.

En Streamlit Dashboard:

- Pestaña **Live (Kafka)**: muestra mensajes en tiempo real y gráficos de distribución.
- Pestaña **Procesado (HDFS)**: visualiza datos agregados por Spark Streaming.

Conclusión

La infraestructura base está montada y los componentes clave están funcionando a nivel prototipo. El sistema está listo para ser expandido y optimizado.